Advising Assistant Project Stage 4

CYBI-3331 Software Engineering & Project Management

GitHub repo: 777isabella/Advising-Assistant

Summary:

During the stage of the project, our team worked to develop the functionality of the Advising Assistant System. We focused on implementing essential features that allow students and faculty to access and interact with academic information. The team implemented features such as transcript access, degree plan viewing, and course prerequisite access, along with the development of a secure login and registration, role-based access, and a functional interface. We have applied Scrum practices and documentation by organizing tasks into a sprint backlog, conducting daily Scrum meetings, and tracking progress toward a product increment. Testing was also conducted on core features to ensure functionality, overall, the team delivered a working prototype that demonstrates the use cases.

Ryan

- Implemented login system
- Developed user registration and logout function
- Implemented role-based access
- Implemented course prerequisite viewing
- Wrote test cases

Isabella

- Developed transcript view for both faculty and students
- Implemented advisee's functionality
- Created UML diagrams for use case
- Worked on building the course recommendation

Ingrid

- Developed degree plan review
- Working on advising report generation
- Prepared Scrum documentation
- Performed system testing and integration debugging

Sprint Backlog:

Task	Assigned To	Status
Implement login system	Ryan	Done
Implement user registration system	Ryan	Done

Implement logout functionality	Ryan	Done
Build recommendation engine	Isabella	Done
Build template website	Isabella	Done
Create transcript view	Isabella	Done
Develop degree plan view	Ingrid	Done
Implement course prerequisite viewing	Ryan	Done
Implement advising report	Ingrid	Done
Write test cases	Ryan	Done
Test	Ingrid	Done
Create UML diagrams	Isabella	Done
Prepare Scrum documentation	Ingrid	Done

Daily Scrum Meeting – March 26, 2026

Sprint: Stage 4 Implementation Sprint

Member: Ryan

- Yesterday: Completed login system and worked on writing test cases
- Today: Continued with role-based access and backend support for prerequisite viewing
- Blockers: None

Member: Isabella

- Yesterday: Finished transcript view feature and built template website
- Today: Working on building the recommendation engine
- Blockers: Logic for course recommendation needs adjustment

Member: Ingrid

- Yesterday: Completed degree plan view
- Today: Working on advising report generation and Scrum documentation
- Blockers: Minor issues during testing

Product Increment:

- User login system
- User logout system
- User registration and logout functionality
- Template website
- Transcript viewing for both faculty and students

- Degree plan viewing
- Course prerequisite viewing through course links
- Test cases for system features

Testing Code/Criteria:

Use Case: Generate Course Recommendation

Goal: Ensure the system correctly recommends courses based on completed courses and prerequisites.

Test Case 1: Valid Recommendation

Given this student information:

- Completed Courses: CYBI1, MATH1
- Degree Plan (for now): CYBI1, CYBI2, MATH1, MATH2
- Prerequisites: CYBI1 is a prerequisite for CYBI2, MATH1 is a prerequisite for MATH2

When:

- Faculty clicks “Generate Recommendation”
- Student clicks “Generate Recommendation”

Then:

- System returns: CYBI2, MATH2

Test Case 2 – No Completed Courses/ Missing Prerequisites

Given:

- Completed: []
- Degree Plan: CYBI1, CYBI2
- Prerequisites: CYB2: {CYBI1}

Then:

- System returns: CYBI1 only

Test Case 3 – All Courses Completed

Given:

- Completed all courses

Then:

- System returns: *empty list* or message: “No remaining required courses.”

Actual Testing Code:

```
from recommendation import RecommendationEngine
engine = RecommendationEngine()

def test_basic_recommendation():
    completed = ["CYBI1", "MATH1"]
    degree_plan = ["CYBI1", "CYBI2", "MATH1", "MATH2"]
    prereqs = {"CYBI2": ["CYBI1"], "MATH2": ["MATH1"]}

    result = engine.generate(completed, degree_plan, prereqs)

    assert result == ["CYBI2", "MATH2"]



def test_missing_prereqs():
    completed = []
    degree_plan = ["CYBI1", "CYBI2"]
    prereqs = {"CYBI2": ["CYBI1"]}

    result = engine.generate(completed, degree_plan, prereqs)

    assert result == ["CYBI1"]
```

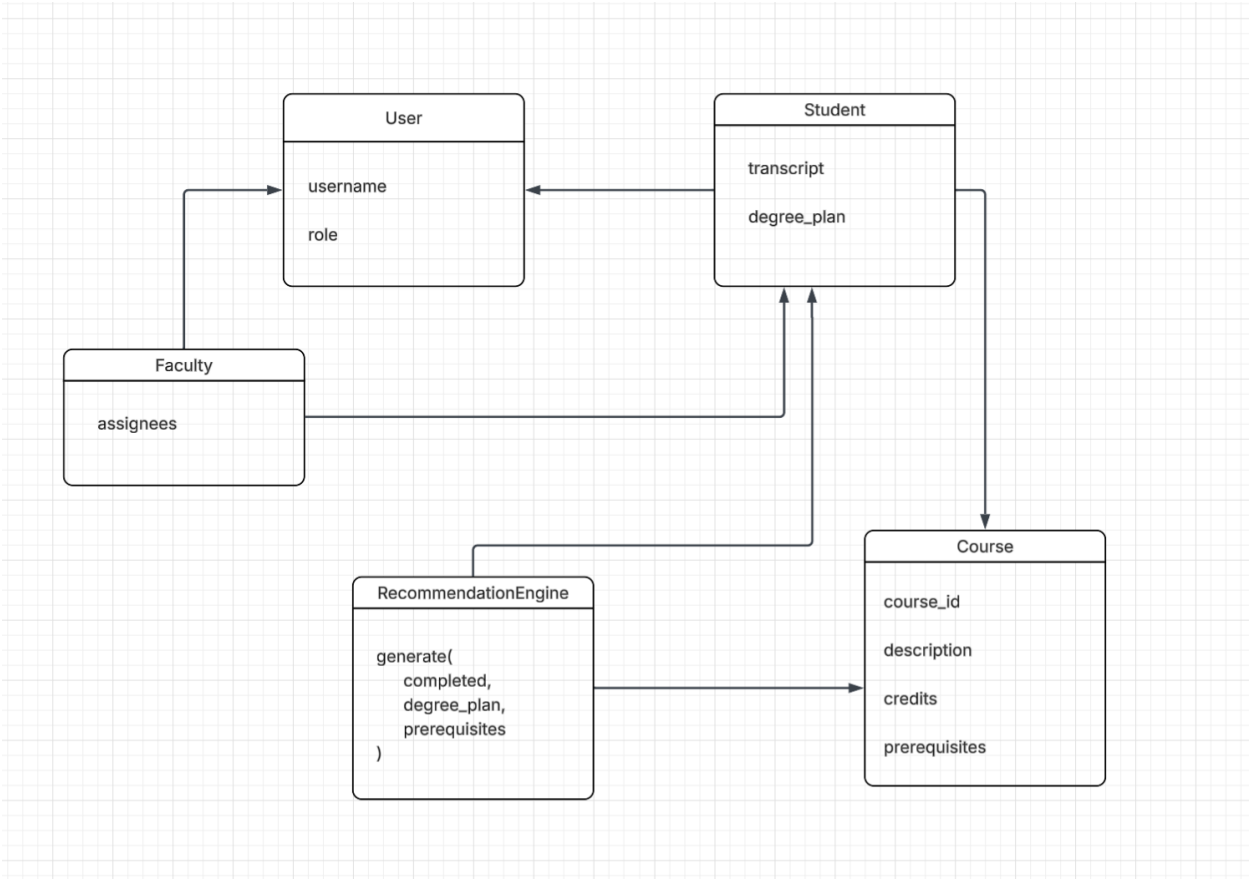
```
def test_all_completed():
    completed = ["CYBI1", "CYBI2"]
    degree_plan = ["CYBI1", "CYBI2"]
    prereqs = {"CYBI2": ["CYBI1"]}

    result = engine.generate(completed, degree_plan, prereqs)

    assert result == []
```

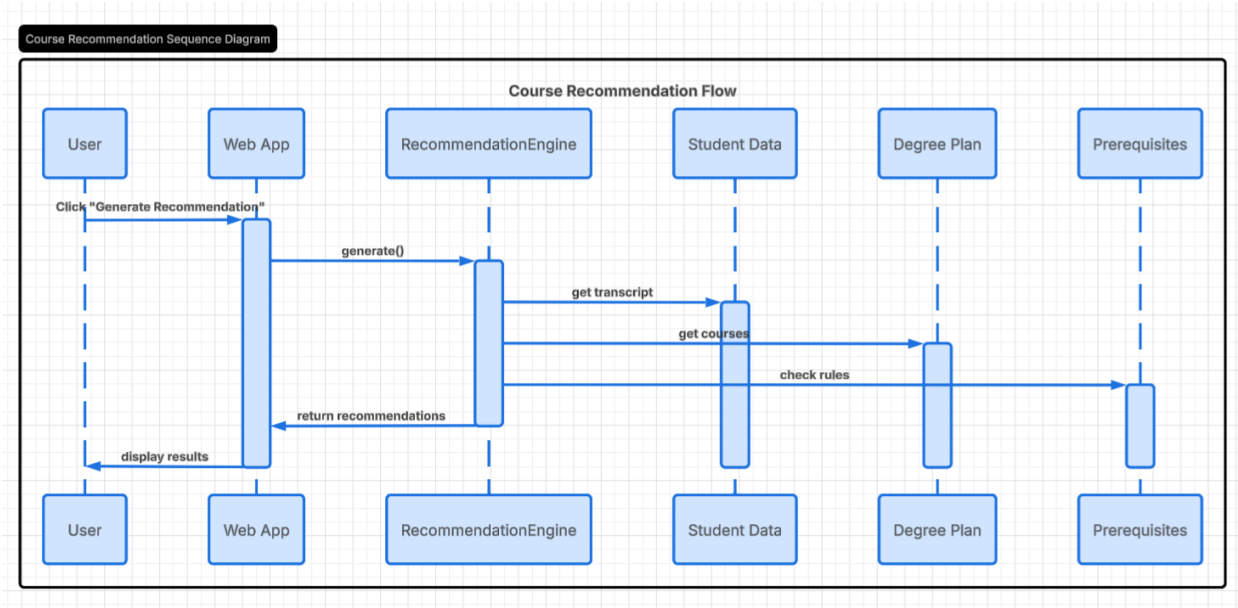
These tests validate that the recommendation engine behaves correctly under different scenarios. We do expect this recommendation engine to become more complicated as we are going to try to implement the UTRGV Cybersecurity degree road map, so the engine will have to not only make recommendations based off prerequisites but also the road map.

UML Diagram:



The relationships here are that the class Student inherits from class User and the attributes of transcript and degree_plan of Student inherit from the Course class. The RecommendationEngine then inherits from the Student and Course classes. Faculty also inherits from the User class and the Student class for the asignees attribute.

Sequence Diagram:



Team Project Peer Evaluation Form for Stage 4

Team Members	Completion of the assigned task (0-20)	Team engagement (0-5)	Justifications for point deductions
Ingrid Alvarez	20	5	N/A
Ryan Saldana	20	5	N/A